

TXSDK OTA 开发指南



保密等级	A	TXSDK OTA 开发指南	文件编号	TX-0000
发行日期	2024/05/07		文件版本	V1.0

责任与版权

责任限制

由于产品版本升级或者其他原因，本文档会不定期更新。除非另行约定，泰芯半导体有限公司对本文档所有内容不提供任何担保或授权。

客户应在遵守法律、法规和安全要求的前提下进行产品设计，并做充分验证。泰芯半导体有限公司对应用帮助或客户产品设计不承担任何义务。客户应对其使用泰芯半导体有限公司的产品和应用自行负责。

在适用法律允许的范围内，泰芯半导体有限公司在任何情况下，都不对因使用本文档相关内容及本文档描述的产品而产生的损失和损害进行超过购买支付价款的赔偿（除在涉及人身伤害的情况中根据适用的法律规定的损害赔偿外）。

版权申明

泰芯半导体有限公司保留随时修改本文档中任何信息的权利，无需提前通知且不承担任何责任。

未经泰芯半导体有限公司书面同意，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。除非获得相关权利人的许可，否则，任何人不能以任何形式对前述软件进行复制、分发、修改、摘录、反编译、反汇编、解密、反向工程、出租、转让、分许可等侵犯本文档描述的享有版权的软件版权的行为，但是适用法禁止此类限制的除外。



珠海泰芯半导体有限公司
Zhuhai TaiXin Semiconductor Co., Limited

珠海市高新区港湾一号科创园港
11 栋 3 楼

版权所有 侵权必究

Copyright © 2023 by TaiXin All Rights Reserved

保密等级	A	TXSDK OTA 开发指南	文件编号	TX-0000
发行日期	2024/05/07		文件版本	V1.0

修订记录

日期	版本	描 述	修订人
2024-05-07	V1.0		TX

保密等级	A	TXSDK OTA 开发指南	文件编号	TX-0000
发行日期	2024/05/07		文件版本	V1.0

目录

TXSDK OTA 开发指南 1

1. 概述1

2. 固件分区1

3. OTA 开发流程2

4. OTA API2

1. 概述

TXSDK 是泰芯半导体发布的 WiFi/音视频系列芯片开发 SDK。本文档介绍了 TXSDK 的 OTA 功能开发方法。

SDK 支持双固件机制，芯片启动时会选择最后更新的固件运行，如果最新的固件启动失败，会切换到另一份固件启动。如果在 OTA 过程中出现异常断电、重启也不会影响芯片启动，芯片会选择另一份固件启动。

如果产品方案选择了单固件，则要谨慎考虑 OTA 操作，避免因 OTA 失败造成设备无法启动。

2. 固件分区

SDK 默认支持双固件，在 device.c 的 ota_fwinfo_get 函数指定了双固件的分区信息，如下图所示：

```
int32 ota_fwinfo_get(struct ota_fwinfo *pinfo)
{
    #if 1
        pinfo->flash0 = &flash0;
        pinfo->flash1 = &flash0;
        pinfo->addr0 = 0;
        pinfo->addr1 = flash0.size / 2;
        return 0;
    #else
        return -1;
    #endif
}
```

```
struct ota_fwinfo {
    struct spi_nor_flash *flash0; //固件 1 的 flash
    uint32 addr0; //固件 1 的分区地址
    struct spi_nor_flash *flash1; //固件 2 的 flash
    uint32 addr1; //固件 2 的分区地址
};
```

可以根据实际情况修改这个函数，调整双固件的分区。固件的分区地址有一定要求：

地址必须是 128K 的整数倍位置（具体芯片可能会有差异，请向 FAE 确认）。

3. OTA 开发流程

OTA 功能开发相对简单，主要的工作是获取固件数据，这由应用程序自行决定。固件数据可以来自网络，例如使用 tcp, http, udp 协议下载固件；也可以来自 SD 卡，例如插卡升级。

获取固件数据后，执行 `libota_write_fw` API 写入固件数据即可。`libota_write_fw` API 在执行过程中会执行 `ota_fwinfo_get` 函数获取固件的分区信息，自动进行固件分区的切换。

4. OTA API

`libota_write_fw` API 是个宏定义，具体执行的 API 和芯片有关。但是应用程序只需要关注 `libota_write_fw` 的参数即可。

```
libota_write_fw(tot_len, off, data, len)
```

- 参数 `tot_len` [in]: `uint32` 类型，固件总长度
 - 参数 `off` [in]: `uint32` 类型，表示当前这笔数据的偏移，偏移应该是不断累加
 - 参数 `data` [in]: `uint8*` 类型，表示当前这笔数据的地址
 - 参数 `len` [in]: `uint32` 类型，表示当前这笔数据的长度
- ❖ 注: `libota_write_fw` API 写入的是固件 bin 文件数据，如果应用程序添加了自定义的头部信息或尾部信息，在调用此 API 时需要去掉添加的头尾部信息。例如通过网络 OTA 时，由于调用此 API 需要知道固件大小 `tot_len`，所以通常需要添加一个自定义头信息用于携带 `tot_len`，在调用此 API 写入 flash，就需要把自定义头信息去掉。